# CTR Predictive Modeling on High-Cardinality Data using LightGBM

## HarvardX Data Science Professional Certificate: PH125.9x

Fidan Alasgarova

08 April 2026

## Executive Summary

Accurate prediction of Click-Through Rate (CTR) enables advertisers to optimize ad targeting, enhance campaign effectiveness, and increase Return On Investment (ROI). This project developed a high-precision CTR predictive model using the Avazu mobile ads dataset. The primary objective was to transform millions of sparse, anonymized data points into actionable behavioral insights.

Due to computational constraints and the requirement for code reproducibility, a strategic subset of the data was utilized. The data was sampled to allow impressions for a full 24-hour cycle (Appendix 1).

The Avazu dataset is characterized by extreme high cardinality and sparsity, with features like device_ip and site_id containing hundreds of thousands of unique identifiers. The Light Gradient Boosting Machine (LightGBM) framework is a powerful model that groups data into smaller "bins" to make processing faster and more efficient. This approach allows the model to find hidden patterns among millions of rows, resulting in highly accurate predictions for ads that are most likely to be clicked. The final model used 127-leaf decision trees to capture complex non-linear relationships, providing a robust framework for real-time click prediction.

The final LightGBM model achieved an AUC of 0.75402, demonstrating superior ranking ability between click and non-click events. The LogLoss of 0.39397 indicates highly calibrated probabilities, which significantly optimize advertising spend by providing the precise accuracy required for competitive real-time bidding. By improving probability calibration, the model minimizes wasted bids on low-propensity bot traffic and overexposed fatigue ads. This ensures capital is allocated toward high-propensity impressions, significantly increasing the Return on Ad Spend (ROAS) and lowering the Cost Per Acquisition (CPA).

By combining feature engineering with the computational efficiency of LightGBM, the model offers a robust and scalable solution for modern ad-tech challenges. It is highly reusable and adaptable to any high-cardinality CTR dataset. Furthermore, by substituting Avazu-specific identifiers with equivalent features, the core logic of tracking user behavior

remains unchanged. Frequency-based filtering methodologies are universal, and the same mathematical principles used to detect repetitive ad exposure can be repurposed to identify product saturation in retail or flag unusual spikes in web traffic. The model can provide a standardized, scalable blueprint for extracting high-precision insights for many large-scale, sparse tabular datasets.

The ensemble model achieves strong performance by effectively merging the matching capabilities of Matrix Factorization (MF) with LightGBM. The MF model treats the relationship between device_model - user proxy and placement - item proxy as a recommendation problem. It decomposes the sparse interaction matrix into 50 latent dimensions to generate an mf_score, which is used as the baseline for a specific device-placement pair. Further, high-cardinality categorical variables are converted into smoothed numerical probabilities using K-fold target encoding to prevent data leakage. Final LightGBM with 63-leaf trees makes the final binary prediction.

By combining latent affinity scores with engineered features, the model achieves a competitive AUC of 0.7475 and a well-calibrated LogLoss of 0.4024. This hybrid architecture is particularly effective for the Avazu dataset, as it captures both broad behavioral trends and subtle user-item interactions. Ultimately, the integration of targeted feature engineering and model ensembling creates a robust, high-precision framework capable of navigating the extreme sparsity of real-world advertising data.

# Introduction

With the rapid expansion of digital technologies, a digital marketing strategy focuses on achieving specific business objectives such as increasing brand awareness, generating leads, driving website traffic, or boosting sales through online channels. As billions of mobile devices are used worldwide each year, mobile marketing has become an increasingly competitive environment for advertisers. To evaluate the effectiveness of digital marketing campaigns, businesses track KPIs such as ROI, conversion rate, CTR, website traffic and social media engagement rate.

CTR is a fundamental metric in digital advertising. It measures the proportion of users who click on an advertisement after viewing it and is calculated as the number of clicks divided by the number of impressions.

$$CTR = \frac{Number\ of\ Clicks}{Number\ of\ Impressions}$$

A high CTR typically indicates effective targeting and engaging content, while a low CTR suggests that improvements are needed in campaign design or audience targeting. Accurate prediction of CTR enables advertisers to optimize ad targeting, enhance campaign effectiveness and increase ROI.

# Project Overview

The Avazu dataset, available on Kaggle, was released in 2015 as part of a competition to predict CTR based on 11 days of advertising data. It contains anonymized features related to user behavior, device characteristics, and advertisement context. While several datasets were considered for the HarvardX IDV project, the author's background in digital marketing made the Avazu dataset particularly appealing. Predicting CTR is a key challenge in digital advertising, and this dataset provides a realistic environment for applying machine learning techniques to a real-world business problem.

Originally, the Avazu dataset contains over 40 million observations and 24 variables, corresponding to more than 40 million rows and 24 columns. Due to its large size (approximately 1.4 GB), it is not practical to load and model the entire dataset on a standard personal computer because of computational and memory constraints. For the purpose of this project, a representative subset (subset) of 1,810,501 rows and 9 selected features was used. This reduced (24.7 MB) subset was uploaded to GitHub (with max limit 25 MB) to enable efficient processing while preserving the essential structure and variability of the original data. Each row represents a single ad impression, and the selected columns correspond to key features relevant for CTR prediction.

Due to the high dimensionality and sparsity of the Avazu dataset, effective data preprocessing and feature engineering are essential for building a robust predictive model. Initial random sampling resulted in temporal bias, as data for early periods dominated the sample. This was corrected by applying proportional sampling (in chunks), ensuring randomness and representation across the full 24-hour period (Refer to Appendix 1).

```
##########################################################################
#  Load subset of Avazu dataset from the Github repository
##########################################################################

# Note: this process could take a couple of minutes

if(!require(tidyverse)) install.packages("tidyverse")
if(!require(recosystem)) install.packages("recosystem")
if(!require(lightgbm)) install.packages("lightgbm")
if(!require(ROCR)) install.packages("ROCR")

library(dplyr)
library(ggplot2)
library(tidyverse)
library(recosystem)
library(readr)

data_url <- "https://raw.githubusercontent.com/natros22/HarvardX-IDV-CTR-Pred
ictor-Avazu/main/avazu_data.csv.gz"

local_file <- "avazu_dataset.csv.gz"
```

```r
options(timeout = 600)

download.file(data_url, local_file, mode = "wb")

set.seed(123)

avazu_subset <- read_csv(local_file, show_col_types = FALSE)
```

# Data Exploratory Analysis

After prepossessing and feature selection, the working subset contains 1,810,501 observations and 9 features.

```r
# Subset dimenstions
dim(avazu_subset)

## [1] 1810501       9
```

This reduced subset enables efficient computation while retaining the most relevant information for modeling CTR.

```r
# Column Names
colnames(avazu_subset)

## [1] "click"        "hour"         "banner_pos"   "site_id"      "app_id"
## [6] "device_model" "C14"          "C17"          "device_ip"
```

The original Avazu dataset was curated to focus on nine key columns, identified as the most influential features for predicting CTR. By narrowing the data to these specific indicators, the subset size was reduced to fit within a 25 MB (Github limit) for reproducibility. This focused approach ensures the model remains computationally efficient and easy to share while retaining the essential information necessary for accurate CTR prediction.

```r
# Number of Duplicates
sum(duplicated(avazu_subset))

## [1] 99255
```

Within the 1,810,501 row subset, 99,255 entries were identified as duplicates across all recorded features. However, these are not considered redundant or erroneous data. In the absence of unique user identifiers, these 'duplicates' represent recurring identical interactions such as a specific device model on the same application being served the same advertisement multiple times. Retaining these rows is critical, as they capture the frequency and density of ad exposure, which are essential components for modeling user behavior and identifying patterns of ad fatigue.

## Click

Sparsity measures the proportion of empty or zero entries within a subset. The Avazu dataset exhibits extreme sparsity because the vast majority of impressions do not result in a click represented by '0'. While the ratio of '1' to total impressions defines the CTR, the high sparsity of categorical features creates a complex modeling environment.

```
# Total impressions
total_impressions <- nrow(avazu_subset)

# Count the non-clicks
non_clicks <- sum(avazu_subset$click == 0)

# Sparsity
sparsity <- non_clicks / total_impressions

sparsity

## [1] 0.8303923
```

Approximately 83% of the observations in the click column are zeros, indicating that most user-ad pairs have no clicks.

```
# Number of Clicks
sum(avazu_subset$click)

## [1] 307075
```

The click is the target dependent variable the model is trying to predict. There are 307,075 click events in the subset.

## Hour

The hour column encodes the exact timestamp of the impression and captures cyclical human behavior. For example, CTR usually spike during evening leisure hours and drop to near zero during early morning hours when only bots are active. The raw hour feature in the Avazu dataset encodes the time of each ad impression in a numeric format that combines both the date and hour (YYYYMMDDHH).

```
# Modified Hour Feature
avazu_subset <- avazu_subset %>%
  mutate(
    hour_num = as.integer(substr(hour, 7, 8)))
head(avazu_subset)

## # A tibble: 6 × 10
##   click     hour banner_pos site_id  app_id   device_model   C14   C17 device_ip
##   <dbl>    <dbl>      <dbl> <chr>    <chr>    <chr>        <dbl> <dbl> <chr>
## 1     0 14102100          0 85f751fd febd1138 d056b4bf     21611  2480 68a9b92c
## 2     0 14102101          0 75f43c5a ecad2386 f143da2c     21684  2496 211f96b8
```

```
## 3     1 14102101          0 85f751fd 9c13b419 0153a639     18093  2060 b5a22be4
## 4     0 14102102          1 cbb01868 ecad2386 158e4944     19251  2201 751ecf08
## 5     1 14102102          0 1fbe01fe ecad2386 8ce34352     15703  1722 f77d21f9
## 6     0 14102100          0 85f751fd 9c13b419 81892725     20146  2306 57e202b1
## # ℹ 1 more variable: hour_num <int>
```

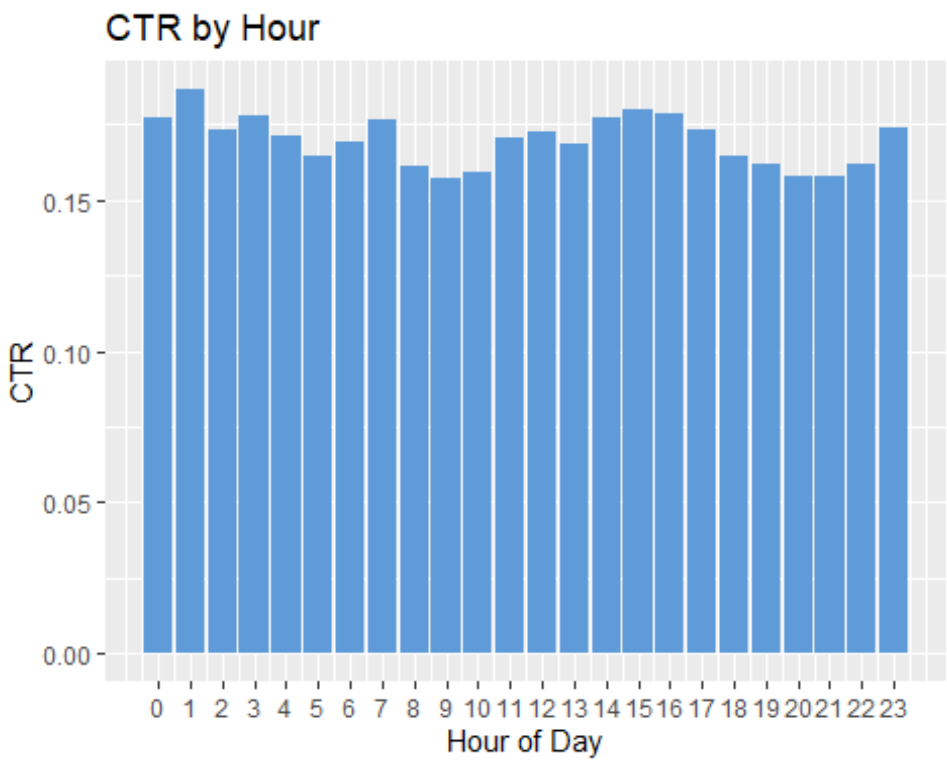
To analyze how the timing of ads affects the CTR, it is necessary to represent the hour of the day in a clear numeric format. The original Avazu dataset was sampled to extract rows that capture ad impressions throughout the full 24-hour cycle, as confirmed by examining the unique values of the hour_num variable.

```r
# Impressions per Hour
table(substr(as.character(avazu_subset$hour), 7, 8))

##
##     00     01     02     03     04     05     06     07     08     09     10
##  37775  44611  56229  63854  78253  86726  78782  85175  96343 104171  96190
##     11     12     13     14     15     16     17     18     19     20     21
##  92037  98525 106853  97058  92395  92103  92367  80083  59017  50877  43855
##     22     23
##  40371  36851
```

The output shows values ranging from 0 to 23, indicating that all hours of the day are represented for a comprehensive analysis of patterns, such as variations in CTR throughout the day. The distribution of impressions by hour shows that user activity fluctuates throughout the day, with some hours exhibiting notably higher ad impressions. These patterns are useful for understanding user behavior and optimizing ad timing.

```r
# CTR by Hour
avazu_subset %>%
  group_by(hour_num) %>%
  summarise(ctr = mean(click)) %>%
  ggplot(aes(x = hour_num, y = ctr)) +
  geom_col(fill = "#5E9BD8") +
  scale_x_continuous(breaks = 0:23) +
  labs(title = "CTR by Hour", x = "Hour of Day", y = "CTR")
```




While the total number of ad impressions changes significantly throughout the day, the CTR stays relatively consistent. This shows that the time of day mainly affects how many people are online, rather than how likely any single person is to click an ad. Since the Avazu dataset covers only 11 days, analysis of weekly and monthly trends, as they lack repetitive patterns is unlikely to produce reliable insights.

## C14 and C17

We make the inference that C14 is ad ID and C17 is ad group ID by analyzing the hierarchy of the unknown features. C17 and C14 represent a one-to-many relationship, as one ad group can have several ad IDs.

## C14

Ad ID is the unique ID for the specific advertisement. This is one of the strongest features. It identifies what is being shown to user. Some ads are simply more attractive than others. To predict future clicks of the specific ad ID the model tracks its historical success.

```
# Number or ad IDs
length(unique(avazu_subset$C14))

## [1] 2220
```

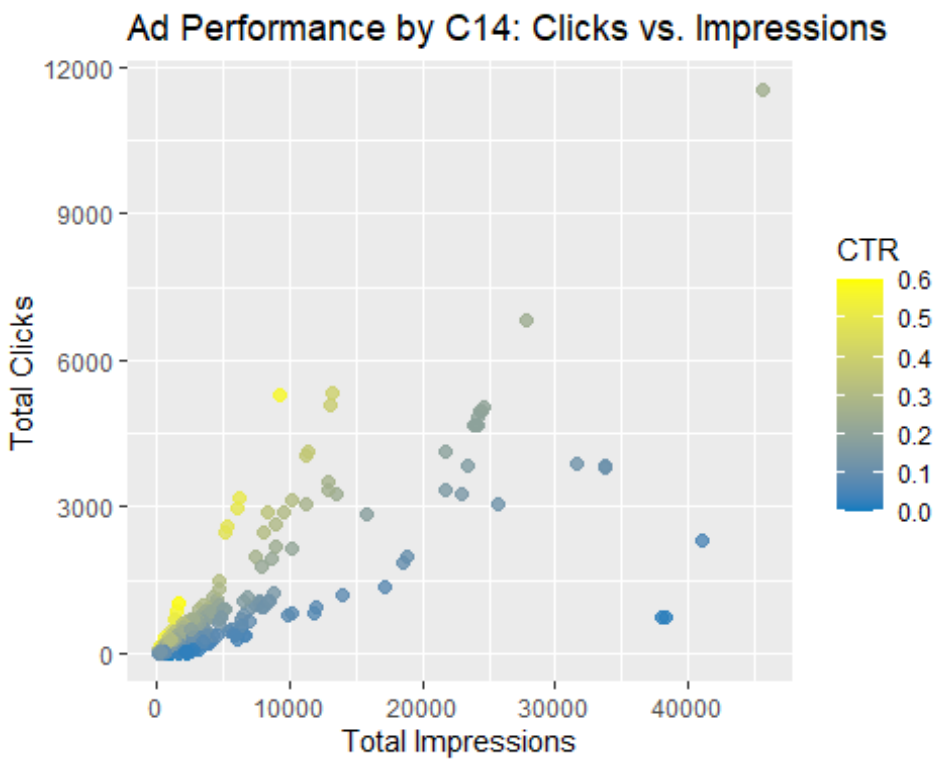
There are 2200 unique ad ID in the subset.

```r
# CTR by ad ID
c14_ctr_analysis <- avazu_subset %>%
  group_by(C14) %>%
  summarise(
    clicks = sum(click),
    impressions = n(),
    ctr = sum(click) / n())
c14_ctr_analysis
```

```
## # A tibble: 2,220 × 4
##      C14 clicks impressions   ctr
##    <dbl>  <dbl>       <int> <dbl>
##  1   375    982        4244 0.231
##  2   376      2           3 0.667
##  3   377    836        3731 0.224
##  4   380    759        3696 0.205
##  5   381     44         192 0.229
##  6   451     13          32 0.406
##  7   452    397        2982 0.133
##  8   454    413        3101 0.133
##  9   455      2           7 0.286
## 10   456    454        3143 0.144
## # ℹ 2,210 more rows
```

The distribution of CTR across different Ad IDs is highly disproportionate. While high-volume ads with low CTRs indicate inefficient spending, niche ads with high CTRs reveal new opportunities for more effective budget allocation. To maintain predictive accuracy, the model must account for the noise such as a deceptive 50% CTR from only two impressions. By smoothing these outliers toward the global average, the model prevents overestimating the performance of rare ads and ensures more reliable predictions.

```r
# Aggregating data by C14
c14_summary <- avazu_subset %>%
  group_by(C14) %>%
  summarise(
    total_impressions = n(),
    total_clicks = sum(click),
    ctr = total_clicks / total_impressions) %>%
  filter(total_impressions > 100)
ggplot(c14_summary, aes(x = total_impressions, y = total_clicks, color = ctr)
) +
  geom_point(alpha = 0.8, size = 2) +
  scale_color_gradient(low = "#007BC0", high = "yellow") +
  labs(
    title = "Ad Performance by C14: Clicks vs. Impressions",
    x = "Total Impressions",
    y = "Total Clicks",
    color = "CTR")
```




High-volume ads with low CTR represent an expensive and inefficient budget allocation, while low-volume ads with high CTR are high-potential opportunities that the model should prioritize.

## C_17

C17 is believed to be the ad group identifier that groups similar ads (C14) into a single campaign category. If a specific new ad has no history yet, the model looks at its ad group to guess how it might perform. This makes it a reliable option, allowing the model to make calculated predictions for new ads by using the past success of their broader group.

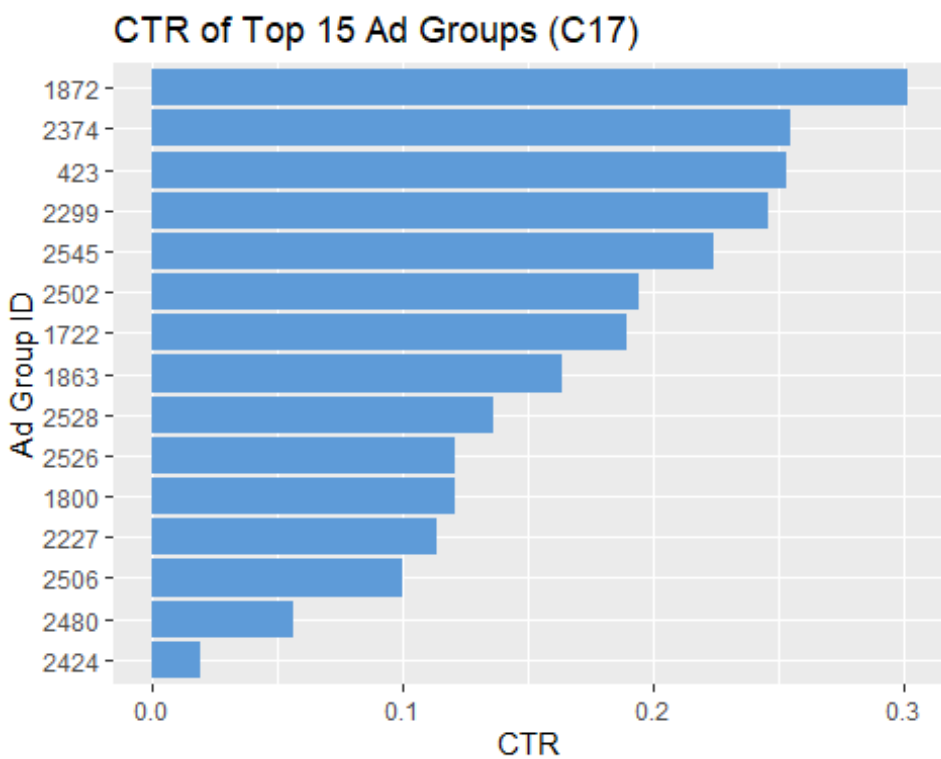
```
# Number of Group IDs
length(unique(avazu_subset$C17))

## [1] 397
```

There are 397 Ad Groups in the subset.

```
# CTR of top 15 Ad Groups
c17_summary <- avazu_subset %>%
  group_by(C17) %>%
  summarise(
    impressions = n(),
    ctr = mean(click),
    unique_ads = n_distinct(C14)) %>%
  arrange(desc(impressions))
c17_summary %>%
```

```
  top_n(15, impressions) %>%
  ggplot(aes(x = reorder(as.factor(C17), ctr), y = ctr)) +
  geom_bar(stat = "identity", fill = "#5E9BD8") +
  coord_flip() +
  labs(title = "CTR of Top 15 Ad Groups (C17)", x = "Ad Group ID", y = "CTR")
```




Some ad groups have distinctly higher CTR than others. By looking at these groups, the model can easily find high-performance clusters that attract much more attention than the average ad, helping it identify which campaigns are naturally more appealing to users.

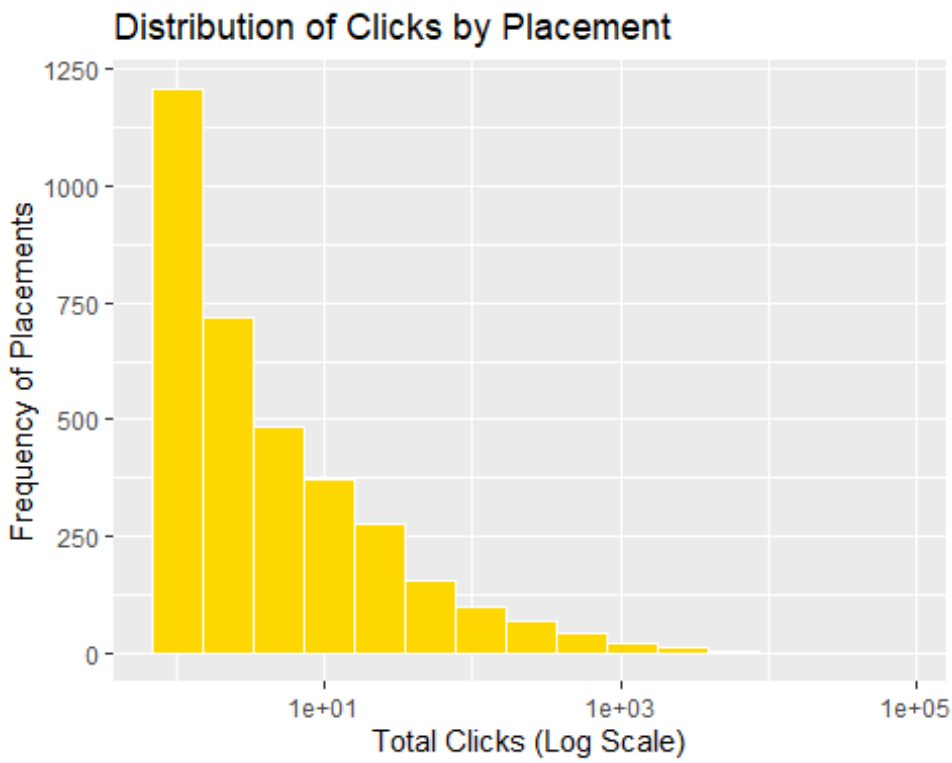
By combining C14 and C17 we can assess the success of the creative of the specific ad. If every C14 performs differently within the same campaign C17, it suggests that the individual ad's design - the creative is the real driver of the click.

## Site ID and App ID

Site ID is the unique ID of the website hosting the ad. It is a high-signal feature for where the interaction happens. App ID is the unique ID of the mobile application hosting the ad and is similar to site_id, but for the app ecosystem. In the Avazu dataset, site_id and app_id serve as the primary environment indicators. To reduce data sparsity and simplify the model, these are combined into a single Placement ID. This consolidated feature captures the specific digital context of each ad impression.

```
# Distribution of Clicks by placement ID
avazu_placement <- avazu_subset %>%
  mutate(placement_id = ifelse(site_id == "85f751fd", app_id, site_id))
```

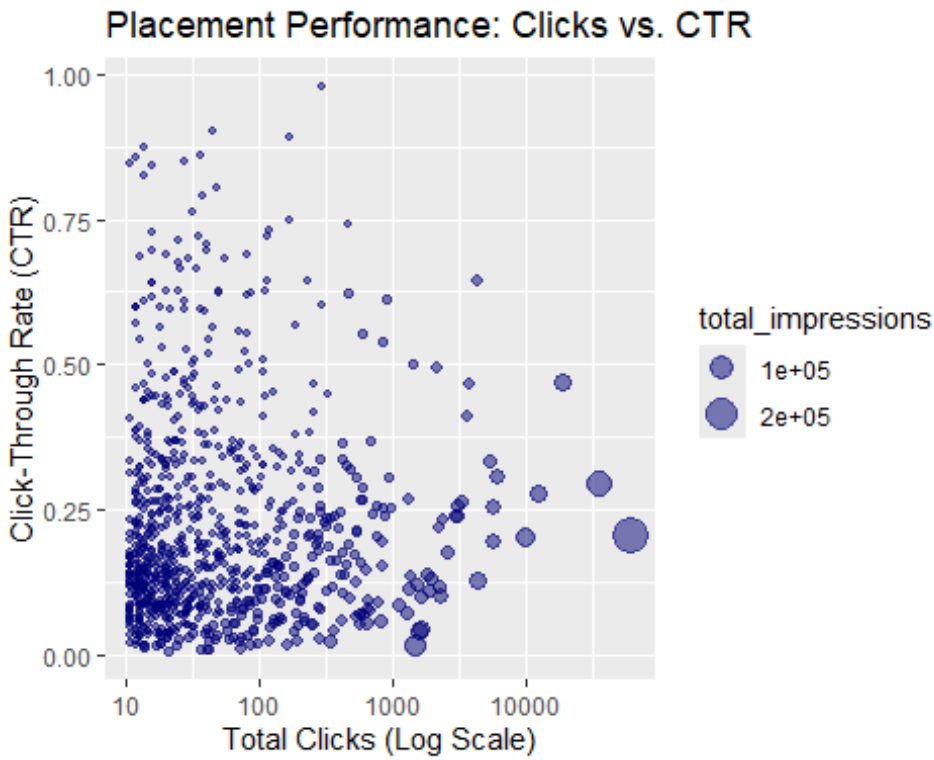
```r
avazu_placement %>%
  group_by(placement_id) %>%
  summarise(total_clicks = sum(click)) %>%
  ggplot(aes(x = total_clicks)) +
  geom_histogram(bins = 15, fill = "gold", color = "white") +
  scale_x_log10() +
  labs(title = "Distribution of Clicks by Placement",
       x = "Total Clicks (Log Scale)",
       y = "Frequency of Placements")
```




The Placement ID analysis reveals a highly skewed distribution, where a small number of high-traffic entities generates the vast majority of total clicks. This concentration of activity suggests that ad engagement is not uniform; rather, it is driven by a few dominant platforms. For predictive modeling, this necessitates a balance between capturing the high-signal patterns of these placements and preventing the model from overfitting to the long-tail of low-volume sites and apps that lack sufficient statistical significance.

```r
# Placement by Clicks VS CTR
placement_performance <- avazu_placement %>%
  group_by(placement_id) %>%
  summarise(
    total_clicks = sum(click),
    total_impressions = n(),
    ctr = total_clicks / total_impressions) %>%
  filter(total_clicks > 10)
ggplot(placement_performance, aes(x = total_clicks, y = ctr)) +
```

```r
  geom_point(aes(size = total_impressions), color = "#007", alpha = 0.5) +
  scale_x_log10() +
  labs(title = "Placement Performance: Clicks vs. CTR",
    x = "Total Clicks (Log Scale)",
    y = "Click-Through Rate (CTR)")
```




The Placement Performance scatterplot illustrates the relationship between engagement volume and efficiency. By plotting total clicks against CTR on a logarithmic scale, we can distinguish between high-volume placements that drive the majority of traffic and high-efficiency niche placements. The bubble size, representing total impressions, further highlights the scale of each environment. This visualization confirms a power-law distribution in the Avazu data, where a small number of placements provide the most significant signal for the model, while the long-tail of smaller placements requires robust regularization to prevent overfitting.

## Device Model

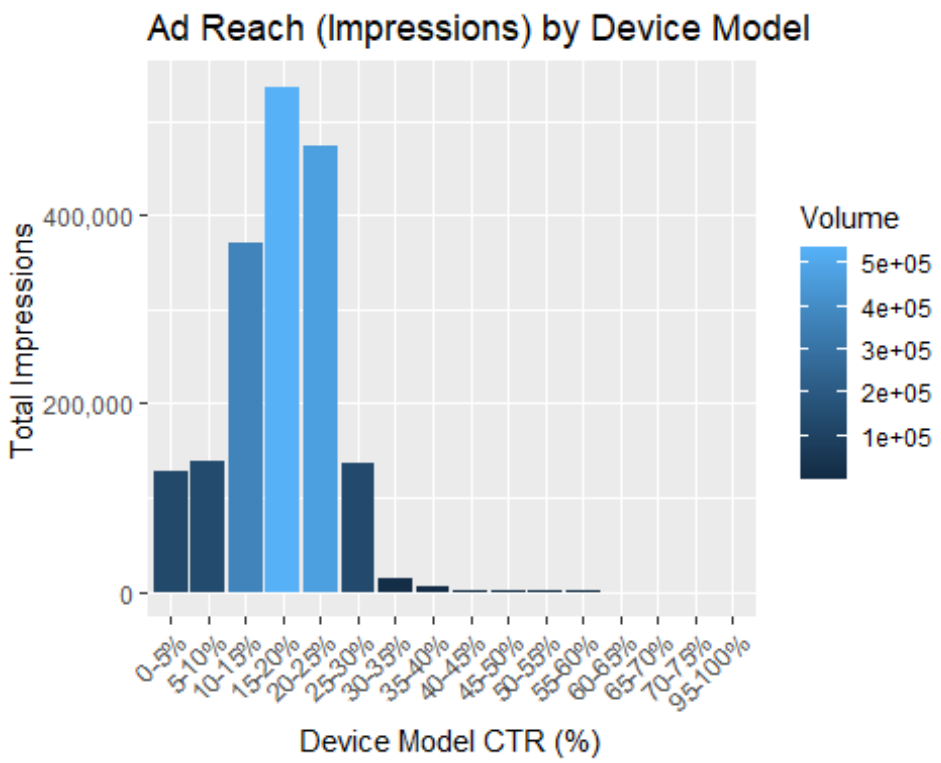
Device model acts as a high-quality demographic proxy, that segments user groups by wealth, technical capability, and behavior. While more expensive models often correlated with higher CTR for luxury goods, travel and financial service, budget models show higher engagement with free-to-pay games and discount offers. Furthermore, screen size and connection speed affect ad's bounce rates, as banner visibility and load times also have effect on CTR.

```r
# Number of unique device models
length(unique(avazu_subset$device_model))

## [1] 5698
```

The subset contains 5,698 unique mobile device models, reflecting the high device diversity of the mobile advertising ecosystem. This high cardinality allows the model to use the device model as a sophisticated demographic proxy for user purchasing power and technical capabilities, which is essential for personalizing ad delivery in an anonymized environment.

```r
# Device Models by CTR
device_tiers <- avazu_subset %>%
  group_by(device_model) %>%
  summarise(impressions = n(), ctr = mean(click)) %>%
  mutate(ctr_tier = cut(ctr,
                        breaks = seq(0, 1, by = 0.05),
                        include.lowest = TRUE,
                        labels = paste0(seq(0, 95, by = 5), "-", seq(5, 100,
by = 5), "%"))) %>%
  group_by(ctr_tier) %>%
  summarise(total_impressions = sum(impressions)) %>%
  filter(!is.na(ctr_tier))
ggplot(device_tiers, aes(x = ctr_tier, y = total_impressions, fill = total_im
pressions)) +
  geom_col() +
  scale_y_continuous(labels = scales::comma) +
  labs( title = "Ad Reach (Impressions) by Device Model", x = "Device Model C
TR (%)", y = "Total Impressions", fill = "Volume") +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```




The distribution of impressions across device model reveals that ad reach is not uniform. While many device models exhibit low engagement, a smaller subset of high-performing devices achieves significantly higher CTR. This disparity confirms that device_model is a high-signal predictor, as the model must distinguish between high-volume devices and high-efficiency models to optimize bid accuracy.

## Device_IP

By tracking specific device IP addresses and analyzing their behavior, the model identifies unique users and distinguishes human interaction from automated bot activity. This granularity enables a sophisticated frequency capping analysis: if an advertisement is served multiple times to the same IP without a conversion, the model learns to reduce the click probability for that specific IP, thereby preventing wasted ad spend on unresponsive audiences.

```
# Number of Device IP
ip_num <- length(unique(avazu_subset$device_ip))
ip_num

## [1] 874418
```

With 874,418 unique device IPs in a subset of 1.8 million rows, the subset exhibits extreme high cardinality and captures a wide range of user behavior.

```
# Device IP with a Click
devices_with_click <- avazu_subset %>%
```

```
  filter(click == 1) %>%
  summarise(devices_with_clicks = n_distinct(device_ip))
devices_with_click

## # A tibble: 1 × 1
##   devices_with_clicks
##                 <int>
## 1              199194
```

The number of devices IPs with at least one click is 199194.

```
(devices_with_click/ip_num)*100

##   devices_with_clicks
## 1            22.78018
```

The analysis reveals that only 22.8% of unique device IP addresses in the 1.8 million row subset ever recorded a click. This suggests that nearly 77% of the total audience remains passive, receiving impressions without interacting with the content.

## Banner position CTR

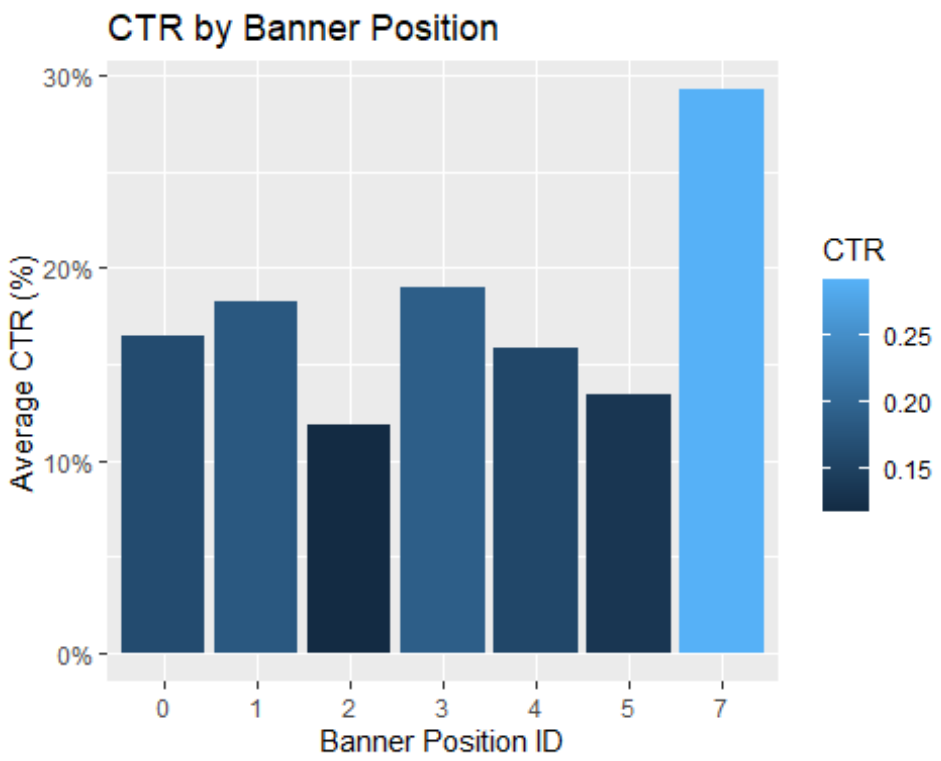
Banner position identifies where on the screen the ad appeared and measures visibility.

```
# Number of Banner Positions
sort(unique(avazu_subset$banner_pos))

## [1] 0 1 2 3 4 5 7
```

There are 7 banner positions available for ads in the subset. The position of the advertisement is an important factor influencing user engagement. Ads placed in more prominent positions are more likely to attract attention and result in higher CTR, while less visible positions tend to generate lower interaction.

```
# Aggregate CTR by Banner Position
banner_ctr <- avazu_subset %>%
  group_by(banner_pos) %>%
  summarise(banner_ctr = mean(click))
ggplot(banner_ctr, aes(x = factor(banner_pos), y = banner_ctr, fill = banner_
ctr)) +
  geom_col() +
  scale_y_continuous(labels = function(x) paste0(round(x*100, 1), "%")) +
  labs(title = "CTR by Banner Position", x = "Banner Position ID", y = "Avera
ge CTR (%)", fill = "CTR")
```




The CTR varies by banner position, highlighting that ad placement influences user engagement. Positions with higher CTR are more effective at generating clicks and should be prioritized in targeting strategies. The highest CTR is attributed to position 7, with position 2 achieving the lowest CTR. Analysis of CTR across different banner positions shows that ad placement significantly affects user interaction with the ad. Certain positions consistently achieve higher CTR, indicating that visibility and placement are key drivers of engagement.

# Evaluation

## Logloss

The Logarithmic Loss (LogLoss) measures the accuracy of predicted probabilities for binary outcomes such as click or non-click (1/0) in CTR predictions. LogLoss is highly sensitive to the calibration of the model, as it measures how closely the predicted probability of a click aligns with the actual outcome. For the Avazu Kaggle competition, LogLoss was the primary evaluation metric because CTR prediction is fundamentally a problem of expected value; advertisers need precise probability estimates to determine the optimal bid for an ad impression. A low LogLoss indicates that the model is not only identifying which users are likely to click, but is also assigning highly reliable probability scores, ensuring that the business can trust these predictions for real-time financial decision-making.

$$\text{LogLoss} = -\frac{1}{n}\sum_{i=1}^{n}[y_i \cdot \log(\hat{y}_i) + (1 - y_i) \cdot \log(1 - \hat{y}_i)]$$

## AUC

Area Under the ROC Curve (AUC) is a performance metric that measures a model's ability to discriminate between classes in this case, whether a user will click an ad or not. Unlike metrics that depend on a specific classification threshold, AUC evaluates the probability that a randomly chosen positive instance (click = 1) is ranked higher by the model than a randomly chosen negative instance (click = 0). For the CTR prediction, AUC is particularly valuable because it assesses the model's ranking proficiency. In the context of digital advertising, accurately identifying which ads are relatively more likely to be clicked is often more critical for effective ad placement and bidding strategies than the absolute probability values themselves, making AUC an essential complement to LogLoss for a robust evaluation.

$$\text{AUC} = \frac{\text{Number of correctly ordered positive-negative pairs}}{\text{Total number of positive-negative pairs}}$$

# LightGBM

The LightGBM model was chosen for this project because of its exceptional ability to handle the scale, sparsity, and high cardinality of the dataset. Specifically, LightGBM was selected because its histogram-based learning and leaf-wise growth strategy allows it to process hundreds of thousands of unique identifiers like site ID, app ID and IP addresses without exhausting system memory or losing accuracy. It excels at finding deep, non-linear interactions between features, such as identifying the exact moment a specific device type on a particular app is most likely to engage with an ad. This makes it a robust, high-speed solution for the "big data" challenges of real-time click prediction.

Training set 80% - used to fit the model and learn patterns in user interactions with ads.
Testing set 20% - held out to evaluate model performance on unseen data, ensuring that results reflect how the model would perform in the real-world scenario.

## Feature Engineering

Hour feature is applied to capture human behavior throughout the day and to identify peak engagement periods when CTR are highest and detects off-peak intervals where automated bot traffic is more prevalent, distinguishing between genuine human activity and machine-generated impressions.

Device_ip and device_model are concatenated to create a user proxy to track behavior in an anonymized environment. This fingerprint is paired with $\log(1 + n)$ frequency counts to flag automated bots or data centers that generate high volumes of impressions without any engagement, allowing the model to drastically lower the predicted probability for those

rows. Additionally, the device_ip provides critical context regarding a user's network speed, distinguishing between high-speed corporate WiFi and slow mobile signals to better estimate the likelihood of an ad successfully loading and being clicked.

Site_id and app_id are utilized as supplementary features and treated interchangeably as a unified placement type. This consolidation reduces feature sparsity and dimensionality, allowing the model to learn the value of the advertising placement more effectively across both mobile web and applications.

C14, inferred to be the ad ID, is combined with the hour feature to monitor hourly impression density and detect ad fatigue. This occurs when an ad is shown too many times, causing it to become repetitive and lose its effectiveness. The model automatically predicts a lower likelihood of clicks due to overexposure.

C17 is believed to represent an Ad Group or Campaign ID. If a specific ad (C14) is new and lacks a performance history, the model uses the aggregate performance of the C17 group to estimate its probability of CTR.

Banner_pos feature serves as a critical measure of visibility, allowing the model to establish a baseline probability before analyzing specific ad content. Since ads at the top of the screen typically yield a much higher CTR than those hidden at the bottom, the model uses this metric to identify which screen placements, when paired with specific sites or apps, drive the highest likelihood of user engagement.

## LightGBM CTR Prediction Machine

```r
library(dplyr)
library(readr)
library(lightgbm)
library(ROCR)

# Ensure reproducibility
set.seed(123)

# Reload avazu_subset
avazu_subset <- read_csv(local_file, show_col_types = FALSE)

# Feature engineering
avazu_subset <- avazu_subset %>%
  mutate(
    # Hour of day
    hour_day = as.integer(substr(as.character(hour), 7, 8)),
    # User Proxy
    user_proxy = paste(device_ip, device_model, sep = "_"),
    # Placement Identity
    placement = ifelse(site_id == "85f751fd", app_id, site_id)) %>%
  # Ad appearances per hour
  group_by(hour, C14) %>%
```

```r
  mutate(ad_hourly_cnt = log1p(n())) %>%
  # IP Frequency
  group_by(device_ip) %>%
  mutate(ip_cnt = log1p(n())) %>%
  ungroup()

# Define feature sets
features <- c("hour_day", "banner_pos", "placement", "device_model",
              "user_proxy", "C14", "C17", "ip_cnt", "ad_hourly_cnt")
categorical_features <- c("banner_pos", "placement", "device_model", "user_pr
oxy", "C14", "C17")
target <- "click"

# Train/Test split
train_idx <- sample(1:nrow(avazu_subset), 0.8 * nrow(avazu_subset))
train_set <- avazu_subset[train_idx, ]
test_set  <- avazu_subset[-train_idx, ]

# Categorical Encoding
for (col in categorical_features) {
  train_levels <- unique(train_set[[col]])

  # Convert to factor based on training levels only
  train_set[[col]] <- as.integer(factor(train_set[[col]], levels = train_leve
ls)) - 1
  test_set[[col]]  <- as.integer(factor(test_set[[col]],  levels = train_leve
ls)) - 1}

# Prepare LightGBM dataset
dtrain <- lgb.Dataset(
  data = as.matrix(train_set %>% select(all_of(features))),
  label = train_set[[target]],
  categorical_feature = categorical_features)

dtest <- lgb.Dataset(
  data = as.matrix(test_set %>% select(all_of(features))),
  label = test_set[[target]],
  reference = dtrain)

# Optimized Parameters
params <- list(
  objective = "binary",
  metric = "binary_logloss",
  boosting = "gbdt",
  learning_rate = 0.05,
  num_leaves = 127,
  feature_fraction = 0.7,
  bagging_fraction = 0.7,
  bagging_freq = 5,
```

```r
  force_col_wise = TRUE,
  verbosity = -1)

# Train model
lgb_model <- lgb.train(
  params = params,
  data = dtrain,
  nrounds = 500,
  valids = list(test = dtest),
  early_stopping_rounds = 30)

# Predict
pred_prob <- predict(lgb_model, as.matrix(test_set %>% select(all_of(features
))))
pred_obj <- prediction(pred_prob, test_set[[target]])

# Evaluate
# Calculate AUC
auc_val <- performance(pred_obj, "auc")@y.values[[1]]

# Calculate LogLoss (with epsilon to avoid Inf)
logloss_val <- -mean(test_set[[target]] * log(pred_prob + 1e-15) +
                   (1 - test_set[[target]]) * log(1 - pred_prob + 1e-15))

# Results
cat("AUC:    ", round(auc_val, 5), "\n")

## AUC:     0.75402

cat("LogLoss:", round(logloss_val, 5), "\n")

## LogLoss: 0.39397
```

## Regularization

To prevent overfitting on features with high cardinality, a feature_fraction of 0.7 was applied, forcing the model to select a random 70% subset of features for each tree. Additionally, a bagging_fraction of 0.7 was utilized to reduce the influence of outliers or noisy data points, such as automated bots generating repetitive clicks. This approach made the model less sensitive to variance in the training data and improved its ability to generalize to unseen users.

## Optimization

```r
# Check the training parameters: 127 leaves decision tree
print(lgb_model$params$num_leaves)

## [1] 127
```

The model was trained using a Binary Logistic Loss objective with a learning rate of 0.05, allowing for slower learning and better generalization. Early stopping was implemented to halt training once the validation LogLoss ceased to improve, ensuring the highest level of generalization. By using Integer Encoding for categorical features and Log Counts for frequency features, the model creates a dense matrix that LightGBM uses to build 127-leaf decision trees, increasing depth for complex interactions. This structural depth enables the model to identify intricate relationships, such as "a premium level device_model on a specific ad placement at 8 PM seeing a fresh C14 ad."

# Ensemble: LightGBM and Matrix Factorization CTR Prediction

The second implements a hybrid ensemble that combines MF with LightGBM. This approach uses MF to capture latent user-item interactions and LightGBM to handle non-linear feature patterns and numerical counts.

The MF model treats the relationship between device_model - user proxy and placement - item proxy as a recommendation problem. It decomposes the sparse interaction matrix into 50 latent dimensions to generate an mf_score, which is used as the baseline for a specific device-placement pair. Further, high-cardinality categorical variables are converted into smoothed numerical probabilities using K-fold target encoding to prevent data leakage. Final LightGBM with 63-leaf trees makes the final binary prediction.

The ensemble model utilized an 80 (train) / 20 (test) random split, consistent with the initial project methodology. By allocating 80% of the data to the training set, the model had sufficient examples for training.

## Feature Engineering

Placement combines site_id and app_id to create a unified identity for where an ad appeared, whether it was on a website or within a mobile application.

Hour_num captures cyclical daily patterns, allowing the model to recognize peak engagement times.

sh_key / sb_key interaction keys are created by joining site_id with hour_num and banner_pos. These capture specific contexts, such as how a site's performance fluctuates at different times or across different layout positions.

ad_hourly_cnt log1p of total ad appearances per hour identifies ad fatigue, signaling when an ad is appearing too frequently and its likelihood of being clicked is beginning to drop.

Target Encoding (_te) uses K-fold smoothed means for high-cardinality features like placement, device_model, and C14 that transform massive ID lists into single numeric probabilities without using excessive memory.

A compatibility score generated by mf_score identifies hidden relationships between specific device types and ad placements.

## Ensemble LightGBM and Matrix Factorization CTR Prediction Model

```
# Ensemble: Matrix Factorization and LightGBM

library(dplyr)
library(readr)
library(recosystem)
library(lightgbm)
library(ROCR)

set.seed(123)
# Load Data
avazu <- read_csv(local_file, show_col_types = FALSE)

# Advanced Feature Engineering (Combining both models)

avazu <- avazu %>%
  mutate(
    # Unified Placement (LightGBM)
    placement = ifelse(site_id == "85f751fd", app_id, site_id),
    hour_num = as.integer(substr(as.character(hour), 7, 8)),

    # Interactions (FM)
    sh_key = paste(site_id, hour_num),
    sb_key = paste(site_id, banner_pos),

    # User Proxy (Device + Ad ID)
    user_ad_proxy = paste(device_model, C14, sep = "_")
  ) %>%
  # Ad Fatigue
  group_by(hour, C14) %>%
  mutate(ad_hourly_cnt = log1p(n())) %>%
  ungroup()

# Train/Test Split
train_idx <- sample(1:nrow(avazu), 0.8 * nrow(avazu))
train <- avazu[train_idx, ]
test  <- avazu[-train_idx, ]

# K-Fold Target
target_encode_kf <- function(train_df, test_df, col, k = 5) {
  train_df$fold <- sample(1:k, nrow(train_df), replace = TRUE)
  new_col <- paste0(col, "_te")
  g_mean <- mean(train_df$click)
  for (i in 1:k) {
    means <- train_df[train_df$fold != i, ] %>%
```

```
      group_by(!!sym(col)) %>%
      summarise(m = (sum(click) + 25 * g_mean) / (n() + 25))
    train_df[[new_col]][train_df$fold == i] <- left_join(train_df[train_df$fo
ld == i, ], means, by = col)$m
  }
  test_means <- train_df %>% group_by(!!sym(col)) %>% summarise(m = (sum(clic
k) + 25 * g_mean) / (n() + 25))
  test_df[[new_col]] <- left_join(test_df, test_means, by = col)$m
  train_df[[new_col]][is.na(train_df[[new_col]])] <- g_mean
  test_df[[new_col]][is.na(test_df[[new_col]])] <- g_mean
  return(list(train = train_df, test = test_df))}

# Encoding
for (f in c("placement", "device_model", "C14", "C17", "sh_key", "sb_key")) {
  res <- target_encode_kf(train, test, f)
  train <- res$train; test <- res$test}

# Latent Interaction from Model 1 & 2 - Matrix Factorization
u_map <- unique(train$device_model)
p_map <- unique(train$placement)
map_id <- function(x, ref) { match(x, ref, nomatch = length(ref) + 1) }

train_mf_data <- data_memory(user_index = map_id(train$device_model, u_map),
                             item_index = map_id(train$placement, p_map),
                             rating = train$click, index1 = TRUE)


r <- Reco()
r$train(train_mf_data, opts = list(dim = 50, niter = 100, lrate = 0.1, verbos
e = FALSE))

train$mf_score <- r$predict(train_mf_data, out_memory())
test_mf_data <- data_memory(user_index = map_id(test$device_model, u_map),
                            item_index = map_id(test$placement, p_map), index
1 = TRUE)
test$mf_score <- r$predict(test_mf_data, out_memory())

# LightGBM
# All engineered features + FM score
features <- c("mf_score", "placement_te", "device_model_te", "C14_te", "C17_t
e",
              "sh_key_te", "sb_key_te", "ad_hourly_cnt", "hour_num", "banner_
pos")

dtrain <- lgb.Dataset(data = as.matrix(train[, features]), label = train$clic
k)

params <- list(
  objective = "binary",
  metric = "binary_logloss",
```

```r
  learning_rate = 0.05,
  num_leaves = 63,
  feature_fraction = 0.8,
  force_col_wise = TRUE)

lgb_model <- lgb.train(params, dtrain, nrounds = 500, verbose = -1)

# Evaluation
final_prob <- predict(lgb_model, as.matrix(test[, features]))
final_prob <- pmin(pmax(final_prob, 1e-15), 1 - 1e-15)

auc_val <- performance(prediction(final_prob, test$click), "auc")@y.values[[1
]]
logloss_val <- -mean(test$click * log(final_prob) + (1 - test$click) * log(1
- final_prob))

cat("AUC:", round(auc_val, 4), "\n")

## AUC: 0.7475

cat("LogLoss:", round(logloss_val, 4), "\n")

## LogLoss: 0.4024
```

## Smoothing and Regularizaiton

Smoothing adjusts the calculated CTR for a feature by bringing it closer to the overall average. If a specific device model appears only twice and happens to have one click, its raw CTR is 50%; smoothing pulls that 50% closer to the global average. Regularization is used by K-fold target encoding to calculate a feature's average using one group of data but apply it to another. This prevents the model from simply "memorizing" the answer for specific IDs. In the LightGBM part of the model, regularization happens via the num_leaves = 63 and feature_fraction = 0.8 settings. These prevent the decision trees from becoming too complex or focusing too heavily on a single dominant feature.

## Optimization

```r
# Check the training parameters: 63 leaves decision tree
print(lgb_model$params$num_leaves)

## [1] 63
```

Optimization is balancing predictive depth with computational efficiency through hyperparameter selection and structural constraints. By setting num_leaves to 63 and a learning rate of 0.05, the LightGBM model is tuned to learn complex patterns incrementally. Additionally, using a feature_fraction of 0.8 ensures the model remains robust by preventing any single feature from dominating the decision-making process. These optimizations, combined with the parallel processing of the Matrix Factorization

component, allow the model to handle millions of rows of sparse data quickly while maintaining the high precision.

# Results

## LightGBM

A LightGBM approach is used to address the high-cardinality and sparsity inherent in the Avazu CTR dataset. By engineering time based fatigue features and user-proxy interactions, the model achieves a refined balance between ranking accuracy (AUC) and probability calibration (LogLoss). The final ensemble uses 127-leaf decision trees to capture complex non-linear relationships, providing a robust framework for real-time click prediction.

The model achieved an AUC of 0.75402, demonstrating a 75.4% probability that the model will correctly rank a random click higher than a random non-click. This strong ranking performance, paired with a LogLoss of 0.39397, indicates that the predicted probabilities are well-calibrated and reliable for advertisement budget allocation. By accurately quantifying click probability, advertisers can utilize this model to significantly optimize their ROI, effectively filtering out bot-driven impressions to ensure their budget is focused on high-intent human users, thereby preventing substantial wasted ad spend. This ensures capital is allocated toward high-propensity impressions, significantly increasing ROAS and lowering the CPA.

## Ensemble LightGBM and Matrix Factorization

The ensemble model achieves strong performance by effectively merging the matching capabilities of MF with LightGBM. By combining latent affinity scores with engineered features, the model achieves a competitive AUC of 0.7475 and a well-calibrated LogLoss of 0.4024. This hybrid architecture is particularly effective for the Avazu dataset, as it captures both broad behavioral trends and subtle user-item interactions. Ultimately, the integration of targeted feature engineering and model ensembling creates a robust, high-precision framework capable of navigating the extreme sparsity of real-world advertising data.

## Conclusion

The LightGBM model outperformed a more complex Ensemble, especially because the target encoding already captured most of the signal from high-cardinality features. This makes LightGBM a more efficient and practical choice for real-time production environments where speed and simplicity are as valuable as raw accuracy.

# Furhter development and adaptation

The architectural framework of the LightGBM model is designed for universal application across sparse, tabular datasets. To reuse this LightGBM framework for a different dataset, the methodology remains identical even as the column names can change. The model is highly portable because it focuses on behavioral logic rather than specific Avazu IDs.

## Sample Portability Methodology

1. Identity Proxy: Replace device_ip + device_model with whatever unique identifiers the new data provides (e.g., User_ID, Cookie_ID, or Hashed_Email). The goal is to recreate a “digital fingerprint” to track user history.
2. Universal Fatigue Logic: The ad_hourly_cnt logic is universal; in a new dataset, simply map this to the primary “Offer” or “Product” ID. It will continue to detect if a specific item is being over-exposed to a user in a short periods of time.
3. Frequency Filtering: The ip_cnt logic can be applied to any high-frequency categorical feature (like Source_URL or Publisher_ID) to automatically flag and discount automated bot or crawler traffic.
4. Parameter Portability: The core LightGBM settings—127 leaves for deep interactions and 0.7 feature fraction for regularization, provide a high-performance baseline for almost any sparse, tabular CTR data.

# Appendix. 1

A memory-efficient sampling method was employed using a chunked callback strategy to process the original Avazu dataset into a sample with a maximum size of 25 MB. By selecting 300,000 row segments and applying proportional sampling, a representative 300,000-row subset was extracted, reducing the data to nine high-signal features. Strategic sampling ensured that the data subset accurately represented the entire 24-hour daily range. This methodology guarantees that the final dataset is both computationally manageable and analytically robust.

```
library(readr)
library(dplyr)
library(curl)
library(stringr)

zip_path <- "C:/Users/Documents/Avazu.zip"
unzip_dir <- tempdir()
unzip(zip_path, exdir = unzip_dir)

all_files <- list.files(unzip_dir, full.names = TRUE)
file_name_train <- all_files[str_detect(all_files, "train") & str_detect(all_
files, ".csv")]

set.seed(123)
sample_size <- 300000
chunk_size <- 300000

# Estimate total rows
total_rows <- 6000000
sampling_fraction <- sample_size / total_rows

collected <- list()

callback <- function(x, pos) {

  # Sample proportionally from each chunk
  sampled_chunk <- x %>%
    slice_sample(prop = sampling_fraction)

  collected[[length(collected) + 1]] <<- sampled_chunk

  return(NULL)}

target_file <- "avazu_data.csv.gz"

read_csv_chunked(
  target_file,
```

```r
  callback = DataFrameCallback$new(callback),
  chunk_size = chunk_size,
  show_col_types = FALSE)

## NULL

avazu_sample <- bind_rows(collected)

avazu_sample <- avazu_sample %>% select(
  click,
  hour,
  banner_pos,
  site_id,
  app_id,
  device_model,
  C14,
  C17,
  device_ip)

write_csv(avazu_sample, "avazu_data.csv.gz")
```

References:

Avazu Kaggle competition page: https://www.kaggle.com/competitions/avazu-ctr-prediction/overview